Vivat — Stack Technique Backend

> ***Version** : Fevrier 2026 **Auteur** : Equipe Backend **Statut** : En developpement actif*

Table des matieres

1. Vue d'ensemble
2. Architecture logicielle
3. Fonctionnalite 1 — Content Acquisition Engine (detail)
4. Fonctionnalites 2-11 — Site public et contributeurs
5. Base de donnees
6. Authentification et securite
7. Intelligence artificielle
8. Cache et performance
9. Paiement
10. Infrastructure et deploiement
11. Packages et dependances
12. Inventaire du code

---

1. Vue d'ensemble

Vivat est une plateforme media automatisee qui **ingere des flux RSS**, **enrichit le contenu par IA**, et **genere des articles SEO originaux** a partir de multiples sources. Le backend expose une API REST complète pour le site public, l'espace contributeur et le dashboard admin.

Stack en un coup d'oeil

Couche	Technologie	Version
Framework	Laravel	12
Langage	PHP	8.3+
Base de donnees	MySQL	8.0
Cache / Queues / Sessions	Redis	7
IA	OpenAI GPT-4o	API
Paiement	Stripe	Payment Intents
Auth API	Laravel Sanctum	4.3
Roles & Permissions	spatie/laravel-permission	6.24
Conteneurisation	Docker Compose	PHP + MySQL + Redis
Queue monitoring	Laravel Horizon	—

---

2. Architecture logicielle

Principes d'architecture

Le backend suit une **architecture MVC enrichie** avec une separation stricte des responsabilites :

```
Requete HTTP
    |
    v
[Middleware] ---- Auth (Sanctum) + Rate Limiting + CORS + Role Check
    |
    v
[Controller] ---- Fin et leger : validation + delegation
    |
    v
[Form Request] -- Validation des donnees entrantes (regles declaratives)
    |
    v
[Service] ------- Logique metier pure (testable, reutilisable)
    |
    v
[Job / Queue] --- Taches lourdes asynchrones (IA, scraping, emails)
    |
    v
[Model] --------- Acces aux donnees (Eloquent, relations, scopes)
    |
    v
[Policy] -------- Autorisation (qui peut faire quoi sur quel objet)
    |
    v
[Resource] ------ Transformation JSON de la reponse API
```

Pourquoi cette architecture ?

Principe	Implementation	Avantage
Controllers fins	Le controller ne contient que la validation et la delegation au service	Lisibilite, testabilite
Services pour la logique metier	`ArticleGeneratorService`, `ArticleSelectionService`, `ContentExtractorService`, `RssParserService`, `RecommendationService`	Reutilisables en CLI, Jobs, API
Jobs pour les taches lourdes	`FetchRssFeedJob`, `EnrichContentJob`, `GenerateArticleJob`	Non-bloquant, retry automatique, monitoring Horizon
Form Requests	`StoreSourceRequest`, `GenerateArticleRequest`, etc.	Validation centralisee, messages d'erreur coherents
Policies	`ArticlePolicy`, `SourcePolicy`, `RssFeedPolicy`	Autorisation declarative, couplage faible
Resources	`ArticleResource`, `CategoryResource`, `SubmissionResource`, etc.	Format JSON maitrise, pas de fuite de donnees

---

3. Fonctionnalite 1 — Content Acquisition Engine

C'est la fonctionnalite principale du backend. Un **pipeline automatise en 5 etapes** qui va de la decouverte d'une information dans un flux RSS a la publication d'un article original genere par IA.

Schema du pipeline

```
Sources (medias)
    |
    v
[Etape 1] INGESTION RSS
    |   FetchRssFeedJob (queue: rss)
    |   RssParserService (parse RSS 2.0 / Atom)
    |   Deduplication SHA-256 (url + title)
    |   -> rss_items (status: new)
    |
    v
[Etape 2] ENRICHISSEMENT IA
    |   EnrichContentJob (queue: enrichment)
    |   ContentExtractorService (scraping HTML)
    |   OpenAI GPT-4o (analyse structuree)
    |   -> enriched_items (lead, headings, key_points,
    |      seo_keywords, primary_topic, quality_score, seo_score)
    |   -> rss_items (status: enriched)
    |
    v
[Etape 3] SELECTION INTELLIGENTE
    |   ArticleSelectionService
    |   Score multi-criteres :
    |     - Fraicheur         25%
    |     - Qualite contenu   25%
    |     - Potentiel SEO     30%
    |     - Diversite sources 20%
    |   Regroupement par sujet (Jaccard >= 20%)
    |   Reasoning : "pourquoi CET article ?"
    |
    v
[Etape 4] GENERATION IA
    |   ArticleGeneratorService / GenerateArticleJob (queue: generation)
    |   Prompt = CategoryTemplate (ton, structure, SEO) + sources enrichies
    |   OpenAI GPT-4o (JSON mode)
    |   -> articles (status: draft, title, content HTML, meta SEO, keywords)
    |   -> article_sources (tracabilite)
    |
    v
[Etape 5] PUBLICATION
        Cycle : draft -> review -> published / rejected / archived
        Condition : quality_score >= 60
        published_at defini automatiquement
```

Detail de chaque etape

Etape 1 — Ingestion RSS

Element	Fichier	Role

Model Source	`app/Models/Source.php`	Media (nom, URL, langue, actif)
Model RssFeed	`app/Models/RssFeed.php`	Flux RSS lie a une source + categorie
Model RssItem	`app/Models/RssItem.php`	Article brut decouvert (titre, URL, hash dedup)
Service	`app/Services/RssParserService.php`	Parse RSS 2.0 et Atom, normalise, genere le hash SHA-256 de deduplication
Job	`app/Jobs/FetchRssFeedJob.php`	Queue `rss` : HTTP GET du XML, parse, cree les RssItem avec deduplication
Scheduler	`bootstrap/app.php`	`RssFeed::dueForFetch()` toutes les 30 min

Mecanismes cles :

- **Deduplication** : hash SHA-256 sur `url + title` → colonne `dedup_hash` (unique). Aucun doublon possible.
- **Statuts** : chaque item commence a `new`, passe a `enriching`, puis `enriched` ou `failed`.
- **Intervalles** : chaque flux a son propre `fetch_interval_minutes` (configurable par source).

Etape 2 — Enrichissement IA

Element	Fichier	Role
Service	`app/Services/ContentExtractorService.php`	Scrape l'URL, extrait le contenu via `<article>`/`<main>`, supprime script/nav/footer
Job	`app/Jobs/EnrichContentJob.php`	Queue `enrichment` : extraction + appel OpenAI pour structurer
Model	`app/Models/EnrichedItem.php`	Contenu structure (1:1 avec RssItem)

Ce que l'IA produit pour chaque item :

- `lead` : resume 1-2 phrases
- `headings[]` : titres H2/H3
- `key_points[]` : 3-7 points cles
- `seo_keywords[]` : 5-10 mots-cles longue traine
- `primary_topic` : sujet principal en 2-4 mots
- `quality_score` : 0-100 (qualite redactionnelle)
- `seo_score` : 0-100 (potentiel SEO)

Protection : Rate limiter OpenAI a 50 requetes/min (`AppServiceProvider`).

Etape 3 — Selection intelligente

Element	Fichier	Role
Service	`app/Services/ArticleSelectionService.php`	Score, regroupe par sujet, selectionne les N meilleurs

Algorithme de scoring (exemple avec 15 items disponibles pour 1 article/jour) :

```
Score total = Fraicheur (25%) + Qualite (25%) + SEO (30%) + Diversite (20%)
```

Critere	Poids	Logique

Fraicheur	25%	< 48h = max, decroit lineairement sur 7 jours
Qualite contenu	25%	quality_score de l'enrichissement + bonus contenu long
Potentiel SEO	30%	Mots-cles longue traine, faible concurrence, termes specifiques
Diversite sources	20%	Multi-sources = +10/source supplementaire (synthese a haute valeur)

Regroupement par sujet : Similarite de Jaccard sur les mots-cles (>= 20% d'intersection = meme sujet). Un article genere a partir de 3 sources sur le meme sujet est plus riche qu'un article mono-source.

Reasoning : Chaque proposition retournee par l'API inclut un texte explicatif : *"Pourquoi generer CET article ?"* — justifie par les scores, les sources, et le potentiel SEO.

Etape 4 — Generation IA

Element	Fichier	Role
Service	`app/Services/ArticleGeneratorService.php`	Construction du prompt, appel OpenAI, parse, create Article
Job	`app/Jobs/GenerateArticleJob.php`	Version async du service (queue `generation`)
Model	`app/Models/CategoryTemplate.php`	Ton, structure, min/max mots, regles SEO par categorie
Model	`app/Models/Article.php`	Article genere (title, slug, content HTML, meta SEO, keywords)

Comment le prompt est construit :

1. **System prompt** = instructions generales + `CategoryTemplate` (ton professionnel, 800-1500 mots, HTML structure H2/H3, style magazine, regles SEO)
2. **User prompt** = pour chaque source enrichie : titre + lead + key_points + texte extrait + mots-cles SEO consolides
3. **Consignes SEO** : densite mots-cles 1-2%, mot-cle principal dans titre + H1 + premier paragraphe + au moins 2 H2

Tracabilite complete : La table `article_sources` lie chaque article genere a ses items RSS d'origine. On peut toujours remonter de l'article publie jusqu'a la source media.

Etape 5 — Publication

- **Cycle de vie** : `draft` → `review` → `published` / `rejected` / `archived`
- **Condition** : `quality_score >= 60` et status `draft` ou `review`
- **SEO** : `meta_title`, `meta_description`, `keywords[]`, `slug`, `reading_time`

Automatisation

Tache	Frequence	Mecanisme
Fetch des flux RSS	Toutes les 30 min	Laravel Scheduler → `FetchRssFeedJob`
Enrichissement IA	Toutes les heures (50 items max)	Laravel Scheduler → `EnrichContentJob`
Snapshot Horizon	Toutes les 5 min	`horizon:snapshot`
Nettoyage jobs echoues	Quotidien	`queue:prune-failed --hours=168`

Monitoring

- **Laravel Horizon** : dashboard temps reel des queues (`rss`, `enrichment`, `generation`, `default`)

- **Pipeline status** : endpoint `GET /api/pipeline/status` (nombre d'items par statut, feeds actifs, derniers jobs)
- **4 superviseurs Horizon** : un par type de queue avec scaling different

---

4. Fonctionnalites 2-11 — Site public et contributeurs

Organisation des routes API

L'API est organisee en **4 niveaux d'acces** :

Prefixe	Auth requise	Role	Fonctionnalites
`/api/public/*`	Aucune	Visiteur	Articles publies, categories, hub pages, recherche, preferences, recommandations, progression lecture
`/api/auth/*`	Aucune (register/login) / Sanctum	Tous	Inscription, connexion, profil
`/api/contributor/*`	Sanctum	Contributeur ou Admin	Soumission d'articles, historique, paiements
`/api/*` (admin)	Sanctum + role admin	Admin	Pipeline, CRUD sources/feeds/articles, moderation, stats, newsletter
`/api/newsletter/*`	Aucune	Visiteur	Inscription/desinscription newsletter

Fonctionnalites implementees

#	Fonctionnalite	Endpoints cles	Status
1	**Content Acquisition Engine**	`pipeline/*`, `articles/generate`	Complet
2	**Auth + Roles**	`auth/register`, `auth/login`, `auth/me`	Complet
3	**API publique articles**	`public/articles`, `public/articles/{slug}`	Complet
4	**Pages Hub categories**	`public/categories/{slug}/hub`	Complet
5	**Recherche et filtrage**	`public/search?q=...&category=...&date_from=...&reading_time_max=...`	Complet
6	**Personnalisation / Preferences**	`public/preferences` (GET/POST, cookie ou auth)	Complet
7	**Recommandations**	`public/recommendations?interests=...&session_id=...`	Complet
8	**Progression de lecture**	`public/reading-progress` (GET/POST)	Complet
9	**Soumission articles (contributeur)**	`contributor/submissions` (CRUD)	Complet
10	**Moderation (admin)**	`submissions/{id}/approve`, `submissions/{id}/reject`	Complet
11	**Newsletter**	`newsletter/subscribe`, `newsletter/unsubscribe`, `newsletter/confirm`	Complet
12	**Paiement Stripe**	`contributor/payments/create-intent`, `payments/{id}/refund`	Complet

Detail des APIs publiques

Recherche (`GET /api/public/search`) :

- Recherche par mot-cle dans titre, excerpt, meta_description
- Filtres : categorie (slug), date_from/date_to, reading_time_max
- Pagination, tri par date de publication

Recommandations (`GET /api/public/recommendations`) :

- Algorithme de scoring : interets utilisateur (40%) + qualite article (25%) + fraicheur (20%) + popularite (15%)
- Exclusion des articles deja lus (> 50% de progression)
- Fonctionne avec ou sans compte (session_id cookie)

Page Hub (`GET /api/public/categories/{slug}/hub`) :

- Description de la categorie
- 3 articles "a la une" (meilleur quality_score)
- 10 articles les plus recents
- Comptage total d'articles publies

---

5. Base de donnees

MySQL 8.0

36 tables organisees en 5 groupes :

Pipeline (11 tables)

Table	Role	PK
`sources`	Medias (nom, URL, langue, actif)	UUID
`categories`	14 categories thematiques	UUID
`rss_feeds`	Flux RSS lies a une source + categorie	UUID
`rss_items`	Articles bruts decouverts (titre, URL, hash dedup)	UUID
`enriched_items`	Contenu structure par l'IA (1:1 avec rss_item)	UUID
`clusters`	Groupes thematiques d'items	UUID
`cluster_items`	Pivot cluster ←→ rss_item	UUID
`articles`	Articles generes (content HTML, SEO, quality_score)	UUID
`article_sources`	Tracabilite article ←→ sources originales	UUID
`category_templates`	Config de generation par categorie (ton, mots, SEO)	UUID
`pipeline_jobs`	Monitoring des jobs (type, statut, erreurs, retries)	UUID

Utilisateurs et auth (6 tables)

Table	Role
`users`	Utilisateurs (UUID, name, email, language, interests, avatar, bio)

`personal_access_tokens`	Tokens Sanctum (API auth)
`roles` / `permissions` / `model_has_roles` / `model_has_permissions` / `role_has_permissions`	Roles et permissions (spatie)
`password_reset_tokens`	Reset mot de passe
`sessions`	Sessions Laravel

Fonctionnalites site (5 tables)

Table	Role
`submissions`	Articles soumis par les contributeurs (draft → pending → approved/rejected)
`payments`	Paiements Stripe (payment_intent, amount, status, refund)
`newsletter_subscribers`	Abonnes newsletter (email, interests min 3, confirm/unsubscribe tokens)
`reading_histories`	Progression de lecture (user_id ou session_id, article_id, progress 0-100)
`user_preferences`	Preferences visiteurs non connectes (session_id, interests, language)

Legacy (5 tables — donnees importees du site existant)

Table	Rows importees
`tbl_cont_pg`	3 756 articles
`tbl_ref`	71 categories/references
`tbl_usr`	3 utilisateurs
`logs`	0 (conserve pour compatibilite)
`cloaked_ip`	0 (conserve pour compatibilite)

Framework Laravel (5 tables)

`cache` , `cache_locks` , `jobs` , `job_batches` , `failed_jobs` , `migrations`

Indexation

Les index sont optimises pour les requetes frequentes :

- `articles` : index sur `status` , `published_at` , `quality_score` , composites `(status, published_at)` et `(category_id, status)` , **full-text** sur `(title, excerpt)`
- `rss_items` : index sur `status` , `dedup_hash` (unique), `feed_id` , `category_id`
- `submissions` : index sur `status` , composite `(user_id, status)`
- `reading_histories` : index sur `(user_id, article_id)` , `(session_id, article_id)`

Coherence des cles primaires

Toutes les tables metier utilisent des UUID (via le trait `HasUuids` d'Eloquent). Cela garantit :

- Pas de collision lors de la fusion de bases
- IDs non predictibles (securite API)
- Compatibilite avec une architecture distribuee future

---

6. Authentification et securite

Authentification API — Laravel Sanctum

Endpoint	Role	Description
`POST /api/auth/register`	Public	Inscription (cree un compte `contributor`)
`POST /api/auth/login`	Public	Connexion (retourne un Bearer token)
`POST /api/auth/logout`	Auth	Revoque le token courant
`GET /api/auth/me`	Auth	Profil utilisateur + roles
`PUT /api/auth/profile`	Auth	Modifier nom, langue, interets, bio

Strategie single-device : a chaque login, les tokens precedents sont revoques.

Roles et permissions — spatie/laravel-permission

Role	Permissions	Cible
admin	Toutes (21 permissions)	Pipeline, CRUD, moderation, stats, newsletter, paiements
contributor	`articles.view`, `submissions.create`, `submissions.view-own`	Soumission d'articles, consultation
Visiteur (non connecte)	—	Lecture articles, recherche, preferences (cookie), newsletter

Protection des routes

```
/api/public/*           → Aucun middleware (acces libre)
/api/auth/register|login → Aucun middleware
/api/auth/logout|me|...  → auth:sanctum
/api/newsletter/*        → Aucun middleware
/api/contributor/*       → auth:sanctum + role:contributor|admin
/api/* (admin)           → auth:sanctum + role:admin
```

Rate limiting

- **API publique** : 60 requetes/minute par IP (ou par user_id si connecte)
- **OpenAI** : 50 requetes/minute (par item, via le RateLimiter Laravel)

Securite supplementaire

- **HTTPS** obligatoire en production
- **Validation serveur** : toutes les donnees entrantes sont validees via Form Requests
- **Policies** : autorisation objet par objet (qui peut publier, modifier, supprimer)
- `.env` dans `.gitignore` (jamais commite)

---

7. Intelligence artificielle

Modele : OpenAI GPT-4o

Usage	Etape	Entree	Sortie
Enrichissement	Etape 2	Texte brut de l'article source	lead, headings, key_points, seo_keywords, primary_topic, quality_score, seo_score
Generation	Etape 4	N sources enrichies + CategoryTemplate	Article original HTML (800-1500 mots), meta SEO, keywords
Selection	Etape 3	Scores des items enrichis	Classement + reasoning

Integration technique

- **Appels via Jobs asynchrones** : jamais d'appel OpenAI dans une requete HTTP synchrone (sauf generation explicite via admin)
- **JSON mode** : les reponses sont structurees en JSON pour un parsing fiable
- **Rate limiter** : 50 req/min pour eviter les erreurs 429
- **Retry** : les jobs echoues sont automatiquement rejoues (3 tentatives par defaut)
- **Prompts SEO** : densite de mots-cles 1-2%, placement dans titre + H1 + premier paragraphe + H2
- **CategoryTemplate** : chaque categorie a ses propres instructions (ton, longueur, regles SEO)

Cout et optimisation

- Seuls les items au statut `new` sont enrichis (pas de retraitement)
- L'enrichissement est limite a 50 items/heure
- Le delai entre les appels est de 3 secondes (evite les pics)

---

8. Cache et performance

Redis

Redis est utilise pour **3 roles** :

Role	Config	Avantage
Cache applicatif	`CACHE_STORE=redis`	Articles, pages hub, resultats de recherche
Files d'attente	`QUEUE_CONNECTION=redis`	Jobs asynchrones (scraping, IA, emails)
Sessions	`SESSION_DRIVER=redis`	Sessions utilisateur rapides

Laravel Horizon

Horizon fournit un **dashboard temps reel** pour monitorer les queues :

Superviseur	Queue	Workers	Role
`rss`	rss	2	Fetch des flux RSS
`enrichment`	enrichment	3	Enrichissement IA
`generation`	generation	2	Generation d'articles
`default`	default	2	Taches generales

Acces : `http://localhost:8000/horizon`

---

9. Paiement

Stripe — Publication ponctuelle (one-time)

Flux de paiement :

```
1. Contributeur soumet un article
2. POST /api/contributor/payments/create-intent
   → Cree un PaymentIntent Stripe (15 EUR par defaut)
   → Retourne client_secret au frontend
3. Frontend confirme le paiement (Stripe.js)
4. POST /api/contributor/payments/confirm
   → Verifie le statut aupres de Stripe
   → Si succeeded : marque le paiement comme "paid"
   → Soumet automatiquement l'article pour validation
5. Admin approuve ou rejette
6. Si rejete : POST /api/payments/{id}/refund
   → Cree un Refund Stripe
   → Marque le paiement comme "refunded"
```

Statuts : `pending` → `paid` → `refunded` (si rejete) / `failed`

Dashboard contributeur : `GET /api/contributor/payments` — historique complet (montant, statut, article associe)

---

10. Infrastructure et deploiement

Docker Compose (developpement)

```
Services :
  app    : PHP 8.3 Alpine + extensions (pdo_mysql, redis, gd, zip, bcmath, intl)
  mysql  : MySQL 8.0 (healthcheck, volumes persistants)
  redis  : Redis 7 Alpine
  phpmyadmin : phpMyAdmin (port 8080)
```

URL	Service
`http://localhost:8000`	Application Laravel
`http://localhost:8000/horizon`	Dashboard Horizon
`http://localhost:8080`	phpMyAdmin

Environnement identique local / staging / prod

- **Docker** garantit que l'environnement est reproductible
- Les variables d'environnement (`.env`) sont les seules differences entre local et prod
- Les migrations et seeders assurent un schema identique partout

Cible production

Element	Choix
Serveur	VPS (DigitalOcean / Hetzner)

Reverse proxy	Nginx + SSL (Let's Encrypt)
Process manager	Supervisor (pour Horizon + Scheduler)
Monitoring	Sentry ou Bugsnag (a installer)
Debug dev	Laravel Telescope (a installer)

---

11. Packages et dependances

Packages installes et utilises

Package	Version	Role	Statut
`laravel/sanctum`	4.3	Auth API par tokens Bearer	Installe et configure
`laravel/horizon`	—	Monitoring et gestion des queues Redis	Installe et configure
`spatie/laravel-permission`	6.24	Roles (admin, contributor) et 21 permissions	Installe et configure
`spatie/laravel-sluggable`	3.7	Slugs SEO automatiques (Submissions)	Installe et utilise
`stripe/stripe-php`	19.3	API Stripe (PaymentIntent, Refund)	Installe et utilise

Packages prevus (a installer selon besoin)

Package	Role	Priorite
`spatie/laravel-sitemap`	Sitemap XML dynamique pour SEO	Moyenne
`laravel/telescope`	Debug en developpement	Basse
`sentry/sentry-laravel`	Monitoring erreurs en production	Haute (avant prod)

---

12. Inventaire du code

Chiffres cles

Element	Nombre
Routes API	94
Tables en base	36
Models Eloquent	17
Controllers API	16
Services	5
Jobs asynchrones	3
Migrations	24
Roles	2 (admin, contributor)
Permissions	21

Liste des Models

`Article`, `ArticleSource`, `Category`, `CategoryTemplate`, `Cluster`, `ClusterItem`, `EnrichedItem`, `NewsletterSubscriber`, `Payment`, `PipelineJob`, `ReadingHistory`, `RssFeed`, `RssItem`, `Source`, `Submission`, `User`, `UserPreference`

Liste des Controllers

`AdminSubmissionController`, `ArticleController`, `AuthController`, `CategoryController`, `CategoryTemplateController`, `ClusterController`, `ContributorSubmissionController`, `NewsletterController`, `PaymentController`, `PipelineController`, `PreferenceController`, `ReadingHistoryController`, `RssFeedController`, `RssItemController`, `SourceController`, `StatsController`

Liste des Services

Service	Responsabilite
`RssParserService`	Parse RSS 2.0 / Atom, deduplication
`ContentExtractorService`	Scraping HTML, extraction contenu
`ArticleSelectionService`	Selection intelligente multi-criteres
`ArticleGeneratorService`	Generation article IA (prompt, parse, create)
`RecommendationService`	Recommandations personnalisees

Liste des Jobs

Job	Queue	Role
`FetchRssFeedJob`	`rss`	Recupere et parse un flux RSS
`EnrichContentJob`	`enrichment`	Extrait le contenu + enrichit via IA
`GenerateArticleJob`	`generation`	Genere un article via IA (async)

Document genere le 9 fevrier 2026. Mis a jour a chaque evolution de la stack.